1. **Approach and tradeoffs**

Currently, each request runs two counts over ~1M rows contributing to the CPU hitting 85% at ~200 requests/min. I would move this computation from per-request to periodic refreshes.

Freshness target: Standing and percentile may lag by about ~5 minutes. The user's own CSR is always live.

Design: A Supabase cron job refreshes two materialized views every 5 minutes:

- **Score distribution**: (scope, score, users_at_or_above)storing cumulative eligible-user counts per CSR and scope (global/continent), plus total eligible users with CSR > 0. Eligibility and continent mapping are applied at refresh time.
- **User snapshot**: one row per eligible user holding their continent and the score they had at refresh time. Allows self-correction against live CSR.

Both views will have unique indexes for REFRESH MATERIALIZED VIEW CONCURRENTLY, allowing reads during refreshes. Both refreshes would then run within a single transaction, committing only if both succeed. This ensures readers see a consistent refresh generation.

Assumption: averageRanks[0] is intended to represent the current season's CSR. Both the live lookup and snapshots explicitly select that season. This proposal is under the assumption that CSR may decrease between refreshes, if it only increases, self-correction isn't required.

Request flow:

1. Read the live CSR from AverageRanks (one-row lookup).
2. Find the smallest score strictly above the live CSR. Its cumulative count gives usersAbove (or 0 if absent).
3. If the user's snapshot record belongs to the requested scope and its score exceeds their live CSR, subtract 1 to exclude their old self
4. Return Standing = usersAbove +1, percentile = usersAbove * 100 / total, and live CSR

Tradeoffs:

- Rankings lag other user's activities by approx. 5 min
- Floating-point CSR can yield nearly as many distinct scores as users, increasing refresh and index maintenance costs. In the real world I would benchmark this proposed implementation with a similar strategy that computes cumulative counts at fixed score boundaries (rank bucket population distribution table) plus an indexed in-bucket count. This trades smaller snapshots for more expensive reads.

The original code uses a global percentile denominator for continent scope and doesn't filter friends. I'd preserve both behaviors initially and flag them for correction.

2. **Rollout and verification**

- **Staging:** build the objects, unique index, and cron job on production-sized data, measuring refresh time and read latency. Verify avg latency < 200ms.

- **Correctness check:** verify the new path against an exact COUNT over the same snapshot, covering ties, score drops, continent membership, missing users, and empty scopes.
- **Shadow mode**: Run both implementations on sampled requests, return the old result, and log differences attributable to snapshot staleness
- **Ramp** 5% -> 25% -> 50% -> 100%, advancing only if latency and DB CPU are in thresholds. The flag provides instant rollback if thresholds are exceeded.
- **Cleanup:** Remove the old path after one clean release cycle

3. Post-launch monitoring and risks

Metrics: endpoint p50/p95 latency (target avg <200ms), DB CPU at peak (baseline 85%), snapshot age, refresh duration and failures, and sampled standing error versus the live count.

Risks**:**

- **Refresh fails silently and data goes stale:** Alert if snapshot age exceeds 15 minutes; retry, and fall back to the live query via the flag.
- **Refresh slows as users grow:** Alert when refresh duration exceeds ~20% of the interval (or 1 minute). Increase the refresh interval or adopt the bucketed approach if benchmarks show excessive refresh costs.